SECURE DIGITAL INVESTIGATION SYSTEM

SDIS — Complete Code Explanation

Hackathon Project Documentation

Language: C++17 | Interface: Command-Line (CLI) | Year: 2026

1. Project Overview

The Secure Digital Investigation System (SDIS) is a command-line C++ application that simulates a real-world law enforcement case management platform. It was built entirely from scratch using the C++ Standard Library — no external frameworks or databases.

The system has two types of users:

- Detective — creates and manages cases, logs evidence, tracks suspects.
- Commander — approves or denies Detective access requests, and monitors all audit logs.

Key capabilities of the system:

- Role-based login with password authentication.
- A request-and-approval workflow before sensitive actions are allowed.
- Hash-based data integrity to detect if evidence has been tampered with.
- A full audit trail: login history and evidence modification history.

Why This Matters

In real digital forensics, maintaining evidence integrity and having a clear chain of custody is legally critical. SDIS demonstrates how those principles can be built into software from the ground up.

2. Headers and Libraries

The program begins by including the standard C++ libraries it needs. No third-party libraries are used.

```
#include <iostream>   // Handles all input/output: cin, cout
#include <string>   // Enables use of the string data type
#include <vector>   // Enables dynamic arrays (vector<T>)
#include <fstream>   // Included for file I/O (not actively used here)
#include <ctime>   // Provides time() and strftime() for timestamps
#include <algorithm>   // Provides find() used for access checking

using namespace std;   // Lets us write 'string' instead of 'std::string'
```

Note on 'using namespace std;'
This line means we don't have to prefix everything with 'std::'. It is convenient for small programs and learning projects, but in large production software it is avoided to prevent name collisions.

3. The Time Function

Every login and every evidence modification is stamped with the current date and time. This is handled by one helper function.

```
string getCurrentTime()
{
    time_t now = time(0);   // Gets current time as seconds since 1 Jan 1970
    char buf[80];   // A character buffer to hold the formatted string
    strftime(   // Formats the time into a human-readable string
        buf,   // Write into buf
        sizeof(buf),   // Maximum characters to write
        "%d-%b-%Y %H:%M",   // Format: day-Month-Year Hour:Minute
        localtime(&now)   // Convert to local timezone
    );
    return string(buf);   // Convert char array to a C++ string and return it
}
```

Example output: 24-May-2026 14:35

How the format string works
"%d-%b-%Y %H:%M" is a pattern: %d = day number, %b = month abbreviation (e.g. May), %Y = 4-digit year, %H = 24-hour hour, %M = minutes. The output looks like: 24-May-2026 14:35.

4. The Hash Function — Evidence Integrity

One of the most important features of SDIS is its ability to detect if evidence has been quietly changed. This is done using a custom hash function — a mathematical fingerprint of text.

```
int simpleHash(string data)
{
    int hash = 0;   // Start with zero
    for(char c : data)   // Loop through every character in the string
    {
        hash += c * 31;   // Multiply the character's ASCII value by 31 and add to
total
    }
    return hash;   // Return the final number — the fingerprint
}
```

How it works in practice:

- When a piece of evidence is first entered, its text is passed to simpleHash() and the resulting number is saved alongside it.
- When a Detective reviews the case later, simpleHash() is called again on the same evidence description.
- If the two numbers match — the evidence is intact. If they differ — the system warns of tampering.

> **Limitation to note**
> The simpleHash() function is simple by design (suitable for a hackathon prototype). It is not cryptographically secure — two different strings could theoretically produce the same number (a 'collision'). In a production system, SHA-256 from a cryptography library would be used instead.

5. Data Structures — What the Program Stores

C++ structs are used to group related data together. Think of each struct as a custom data type — like a form with fields. SDIS defines six structs that model every entity in the system.

5.1 — Evidence

```
struct Evidence
{
    string description;   // The text description of this piece of evidence
    int integrityHash;   // The hash fingerprint — saved when evidence is first
entered
};
```

Every time evidence is added to a case, its description is stored here along with its hash. If someone edits the description later, the hash will no longer match — revealing the tampering.

5.2 — Suspect

```
struct Suspect
{
    string name;   // The suspect's full name
    string reason;   // Why this person is a suspect
    int score = 0;   // A suspicion score — starts at 10 when added to a case
};
```

5.3 — Case

```
struct Case
{
    string caseNumber;   // Unique ID for this case, e.g. CAS-2026-001
    string crimeType;   // Category of the crime, e.g. Theft, Fraud
    string summary;   // A written description of the incident
    string createdBy;   // The username of the Detective who created it
    vector<Evidence> evidenceList;   // All pieces of evidence for this case
    vector<Suspect> suspects;   // All persons of interest
    int caseIntegrity;   // Hash of the core case fields at creation time
};
```

The Case struct is the central entity. It contains vectors (dynamic arrays) of Evidence and Suspect structs inside it — so each case owns its own list of evidence and suspects.

5.4 — Request

```
struct Request
{
    string username;   // Which detective sent this request
    string action;   // What they are requesting: CREATE CASE or REVIEW CASE
    string time;   // When the request was submitted
};
```

When a Detective needs access to create or review cases, they submit a Request. It gets queued up for the Commander to approve or deny.

5.5 — LoginRecord

```
struct LoginRecord
{
    string username;   // Who logged in
    string role;   // Their role: DETECTIVE or COMMANDER
```

```
    string time;    // When they logged in
};
```

5.6 — TamperRecord

```
struct TamperRecord
{
    string username;    // Who made the change
    string caseNumber;    // Which case was affected
    string action;    // What they did, e.g. MODIFIED EVIDENCE
    string time;    // When it happened
};
```

Important
TamperRecord is created automatically every time a Detective uses the 'Modify Evidence' feature. This means even legitimate edits are recorded. The Commander can review this log at any time to see a full history of changes.

6. Global Data — The Program's Memory

All data in SDIS lives in global variables. Because there is no database or file storage, everything exists only for the duration of the running program.

```
vector<Case>         cases;    // All cases created in this session
vector<Request>      pendingRequests;    // Requests waiting for Commander approval
vector<LoginRecord>  loginHistory;    // Every login that has occurred
vector<TamperRecord> tamperHistory;    // Every evidence modification
vector<string>       approvedCreateUsers;    // Detectives allowed to create cases
vector<string>       approvedReviewUsers;    // Detectives allowed to review cases

string currentUser = "";    // Username of whoever is logged in right now
string currentRole = "";    // Their role: DETECTIVE or COMMANDER
```

Limitation
Because the data is stored in memory only, everything is lost when the program exits. A real investigation system would write this data to an encrypted database so records persist across sessions.

7. Authentication — Logging In

SDIS has two separate login functions — one for Detectives and one for Commanders. Both work the same way: ask for a username and password, check the password, and if correct, set the global user state.

7.1 — Detective Login

```
bool detectiveLogin()
{
    string username, password;
    cout << "Enter username: ";
    getline(cin, username);   // Read the full username including spaces
    cout << "Enter password: ";
    getline(cin, password);

    if(password == "Detective#Secure2026")   // Check against the hardcoded
password
    {
        currentUser = username;   // Save username to global state
        currentRole = "DETECTIVE";   // Save role to global state
        recordLogin(username, "DETECTIVE");   // Add to login audit log
        return true;   // Signal success to main()
    }
    return false;   // Wrong password — deny access
}
```

The Commander login works identically but uses the password "Commander#Secure2026" and sets currentRole to "COMMANDER".

Security Note
Hardcoding passwords directly in source code is not safe for real software — anyone with access to the source file can see them. In production, passwords would be stored as bcrypt hashes in a separate secure configuration file.

8. Access Control — Who Can Do What

Two simple functions check whether a given username has been granted a particular permission by the Commander. They search the relevant approved-users vector and return true or false.

```
bool hasCreateAccess(string username)
{
    return find(   // Search the vector...
        approvedCreateUsers.begin(),   // ...from the start...
        approvedCreateUsers.end(),   // ...to the end...
        username   // ...for this username
```

```
    ) != approvedCreateUsers.end();   // If found, return true; if not found,
return false
}
```

hasReviewAccess() works the same way but checks approvedReviewUsers instead.

These functions are called at the start of createCase() and reviewCase() like this:

```
void createCase()
{
    if(!hasCreateAccess(currentUser))   // If this detective has NOT been
approved...
    {
        cout << "Access not granted yet.";   // ...show an error and stop
        return;
    }
    // ... rest of case creation logic ...
```

9. Requesting Permission

Since Detectives have no access by default, they must request it. This function creates a Request object and adds it to the pending queue.

```
void requestCreateAccess()
{
    Request r;   // Create a new Request object
    r.username = currentUser;   // Set who is requesting
    r.action   = "CREATE CASE";   // Set what they want
    r.time     = getCurrentTime();   // Timestamp it
    pendingRequests.push_back(r);   // Add it to the queue
    cout << "CREATE CASE request sent.";
}
```

requestReviewAccess() works the same way, setting the action to "REVIEW CASE" instead.

10. Creating a Case

This is the most complex function in the Detective panel. It builds an entire Case object interactively — prompting the user for each field, then saving the case to the global cases vector.

```
void createCase()
```

```
{
    if(!hasCreateAccess(currentUser)) { ... return; }   // Permission check first —
exit if not approved

    Case c;   // Create a new empty Case object
    c.createdBy = currentUser;   // Record which detective is creating it

    getline(cin, c.caseNumber);   // Ask for and read the case number
    getline(cin, c.crimeType);   // Ask for and read the crime type
    getline(cin, c.summary);   // Ask for and read the summary
```

Adding Evidence Items

```
    int evidenceCount;
    cin >> evidenceCount;   // Read how many evidence items there will be
    cin.ignore();   // Clear the newline character from the input buffer

    for(int i = 0; i < evidenceCount; i++)
    {
        Evidence e;   // Create a new Evidence object
        getline(cin, e.description);   // Read the evidence text
        e.integrityHash = simpleHash(e.description);   // Calculate and save its
hash fingerprint
        c.evidenceList.push_back(e);   // Add it to the case
    }
```

Adding Suspects

```
    for(int i = 0; i < suspectCount; i++)
    {
        Suspect s;   // Create a new Suspect object
        getline(cin, s.name);   // Read the suspect's name
        getline(cin, s.reason);   // Read why they are a suspect
        s.score = 10;   // All suspects start with a score of 10
        c.suspects.push_back(s);   // Add to the case
    }
```

Finalizing the Case

```
    c.caseIntegrity = simpleHash(   // Hash the core fields together...
        c.caseNumber +   // ...combining case number...
        c.crimeType  +   // ...crime type...
        c.summary   // ...and summary into one string to hash
    );
    cases.push_back(c);   // Add the completed case to the global list
```

Why is the hash calculated at the end?
The integrity hash is a snapshot of the case's core fields at the moment it is created. If anyone later changes the caseNumber, crimeType, or summary directly in memory, the hash will no longer match when the case is reviewed — exposing the tampering.

11. Reviewing a Case — Integrity Verification

The review function displays case details and runs integrity checks on both the case-level fields and each individual piece of evidence.

```
    int verifyCase = simpleHash(   // Re-calculate the hash right now...
        c.caseNumber + c.crimeType + c.summary
    );

    if(verifyCase != c.caseIntegrity)   // Compare to the saved hash from creation
time
    {
        cout << "WARNING: CASE TAMPERING DETECTED!";   // Alert if they differ
    }
```

```
    for(auto &e : c.evidenceList)   // Loop through every piece of evidence
    {
        cout << e.description;   // Display the evidence
        if(simpleHash(e.description) != e.integrityHash)   // Check its hash
        {
            cout << "WARNING: EVIDENCE TAMPERED!";   // Alert if hash does not
match
        }
    }
```

The review function also only shows cases that were created by the currently logged-in Detective, enforcing ownership separation between different detectives.

12. Modifying Evidence — With Audit Trail

Detectives are allowed to update evidence descriptions. However, every modification is automatically written to the tamper history — so the Commander always knows what changed.

```
    getline(cin, c.evidenceList[evidenceChoice - 1].description);   // Overwrite
the evidence description with the new text

    TamperRecord t;   // Create a new tamper record
    t.username   = currentUser;   // Who made the change
    t.caseNumber = c.caseNumber;   // Which case was affected
    t.action     = "MODIFIED EVIDENCE";   // What action was taken
    t.time       = getCurrentTime();   // When it happened
    tamperHistory.push_back(t);   // Add to the permanent audit log
```

Important design choice
Notice that the evidence hash (integrityHash) is NOT updated when evidence is modified. This is intentional — it means that the next time the case is reviewed, the system will detect that the evidence no longer matches its original hash and will display a WARNING. This creates a detectable paper trail of changes.

13. Commander Features

The Commander has three capabilities: reviewing pending requests, viewing login history, and viewing the tamper history.

13.1 — Approving/Denying Requests

```
    if(decision == 'Y' || decision == 'y')
    {
        if(selected.action == "CREATE CASE")
            approvedCreateUsers.push_back(selected.username);   // Add to the
create-access list
        else if(selected.action == "REVIEW CASE")
            approvedReviewUsers.push_back(selected.username);   // Add to the
review-access list
    }
    pendingRequests.erase(pendingRequests.begin() + (choice - 1));   // Remove the
request from the queue
```

13.2 — Login History

```
void viewLoginHistory()
{
    for(auto &l : loginHistory)   // Loop through every recorded login
    {
        cout << l.username   // Print username, role, and time
             << " (" << l.role << ") "
             << "logged in at "
             << l.time << endl;
    }
}
```

13.3 — Tamper History

```
void viewTamperHistory()
{
    for(auto &t : tamperHistory)   // Loop through every recorded modification
    {
        cout << t.username << " " << t.action
             << " in case " << t.caseNumber
             << " at " << t.time << endl;
```

```
    }
}
```

14. Menu Systems — How Navigation Works

Both the Detective and Commander have their own menu loops. They use a do-while loop, which means the menu always runs at least once and keeps repeating until the user chooses to logout.

```
void detectiveMenu()
{
    int choice;
    do
    {
        cout << "1. Request Create Access";   // Display menu options
        cout << "2. Create Case";
        cout << "3. Request Review Access";
        cout << "4. Review Case";
        cout << "5. Modify Evidence";
        cout << "6. Logout";

        cin >> choice;   // Read user's selection
        cin.ignore();

        switch(choice)   // Call the matching function
        {
            case 1: requestCreateAccess(); break;
            case 2: createCase();          break;
            // ... etc ...
        }
    }
    while(choice != 6);   // Keep looping until the user selects Logout (option 6)
}
```

15. The main() Function — Entry Point

main() is where every C++ program starts. In SDIS, it runs the top-level loop that lets the user pick a role to login as, or exit the program entirely.

```
int main()
{
    int choice;
    do
    {
```

```
        cout << "1. Login as Detective";
        cout << "2. Login as Commander";
        cout << "3. Exit";

        cin >> choice; cin.ignore();

        switch(choice)
        {
            case 1:
                if(detectiveLogin())    // Try to login — returns true if password
is correct
                    detectiveMenu();    // If successful, show the detective's menu
                break;

            case 2:
                if(commanderLogin())
                    commanderMenu();
                break;
        }
    }
    while(choice != 3);    // Keep running until user selects Exit

    return 0;    // Program ended successfully
}
```

16. Full Feature Summary

Feature	Role	How It Works
Login	Detective / Commander	Password check; sets currentUser and currentRole
Request Create Access	Detective	Creates a Request and adds it to pendingRequests queue
Request Review Access	Detective	Same as above but for review access
Approve/Deny Request	Commander	Adds username to approvedCreateUsers or approvedReviewUsers
Create Case	Detective (approved)	Builds a Case struct; hashes evidence and core fields
Review Case	Detective (approved)	Recomputes hashes and compares — alerts on mismatch
Modify Evidence	Detective	Updates evidence text; logs a TamperRecord automatically

View Login History	Commander	Prints all LoginRecord entries
View Tamper History	Commander	Prints all TamperRecord entries

17. Conclusion

SDIS demonstrates several important software engineering principles in a clean, self-contained C++ program:

- Role-based access control — different users have different capabilities.
- A request-and-approval workflow — power is not given freely; it must be granted.
- Data integrity verification — hash fingerprints reveal tampering automatically.
- Automatic audit logging — every login and modification is permanently recorded.
- Ownership enforcement — Detectives can only see their own cases.

While the system is built as a prototype (in-memory storage, simple hash, hardcoded passwords), the architecture and logic reflect real principles used in forensic and law enforcement software systems.

Built for the Hackathon
This project was written entirely in C++ using only the Standard Library. No external frameworks, databases, or third-party code. Every feature — from login to tamper detection — was implemented from scratch.

Project Video Submission

Please click this link below to watch our video presentation/project pitch:

https://drive.google.com/file/d/1B-mX4U-uHA9Uz5xcgFfu9EpBDtl5Lojx/view?usp=sharing